

# Module 14 - chapter 11

## Automate restoring EC2 Volume from the Backup

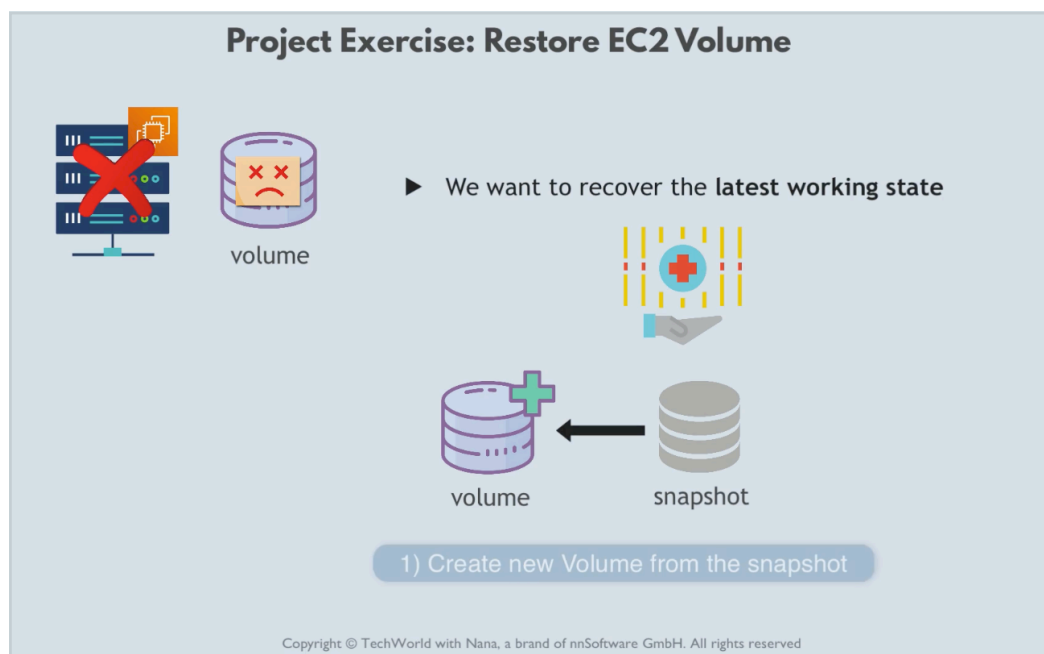
The project file used in this lecture can be found here:

- <https://gitlab.com/twn-devops-bootcamp/latest/14-automation-with-python/automation-projects/-/blob/main/restore-volume.py>

## Restore EC2 Server Data

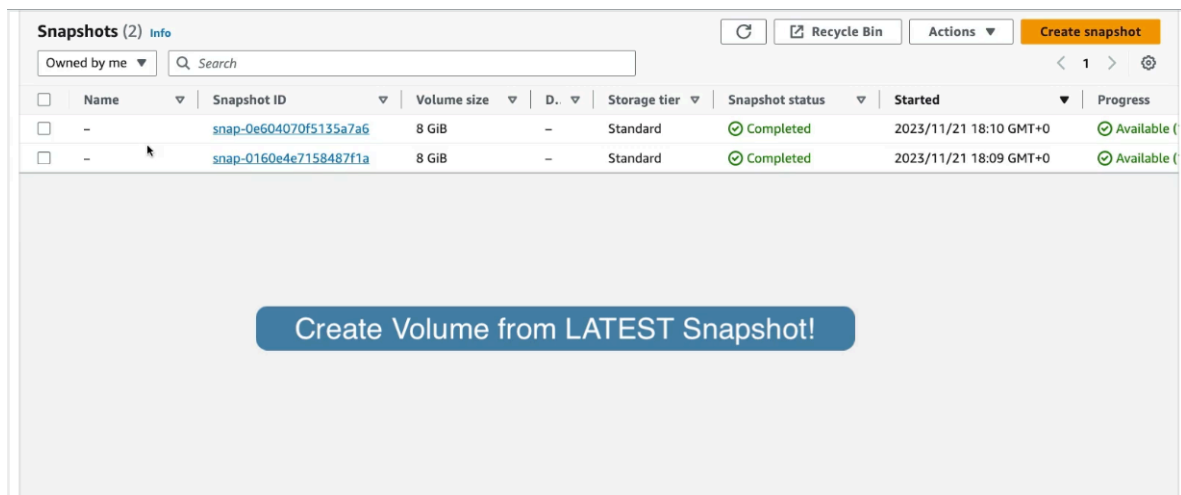
### Project intro

Let's use the snapshot. Let's say our server is crashing due to corrupted data. So we want to recover the latest working state and for that we create a new volume using the snapshot.



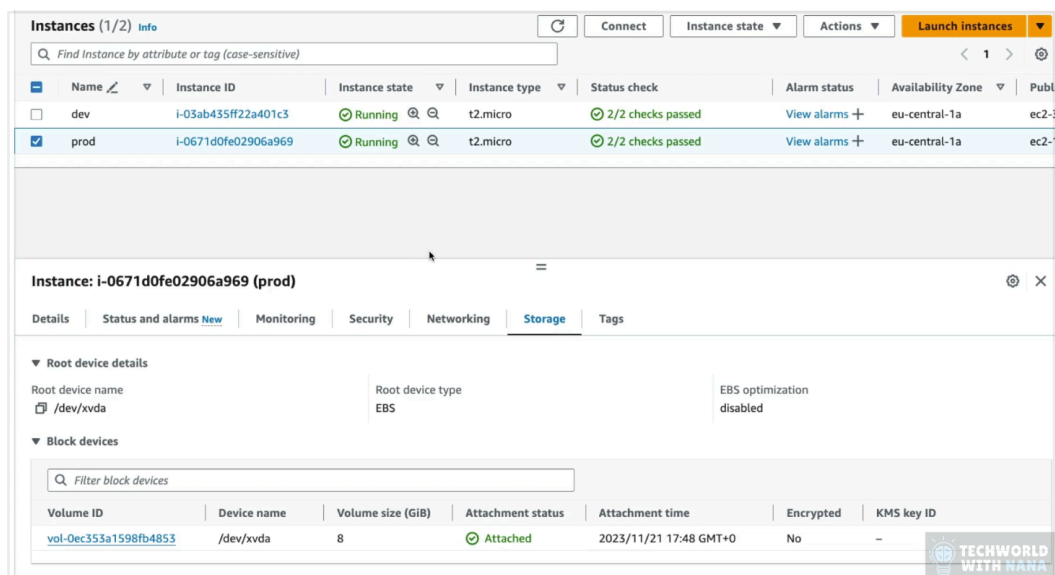
| Volumes (1/2) Info                  |      |  |                                        |      |       |      |            |                 |                        |
|-------------------------------------|------|--|----------------------------------------|------|-------|------|------------|-----------------|------------------------|
| Search                              |      |  |                                        |      |       |      |            |                 |                        |
|                                     | Name |  | Volume ID                              | Type | Size  | IOPS | Throughput | Snapshot        | Created                |
| <input checked="" type="checkbox"/> | prod |  | <a href="#">vol-0ec353a1598fb4853</a>  | gp3  | 8 GiB | 3000 | 125        | snap-08b2fd6... | 2023/11/21 17:48 GMT+0 |
| <input type="checkbox"/>            | -    |  | <a href="#">vol-0addda1a8b3b930319</a> | gp3  | 8 GiB | 3000 | 125        | snap-08b2fd6... | 2023/11/21 17:48 GMT+0 |

So we take the last snapshot to create a new volume



And then attach it to the instance.

Currently the instance has the original volume attached to it. We will need to attach the new volume here:



## Implementation

So we have two servers and one has corrupted data and is now crashing. So we need to access that EC2 instance and its Instance Id

| Instances (1/2) <a href="#">Info</a>     |      |                                                    |                |
|------------------------------------------|------|----------------------------------------------------|----------------|
| Saved filter sets<br>Choose filter set ▼ |      | Find Instance by attribute or tag (case-sensitive) |                |
| <input type="checkbox"/>                 | Name | Instance ID                                        | Instance state |
| <input type="checkbox"/>                 | dev  | i-0b874cf60fbd56050                                | Running        |
| <input checked="" type="checkbox"/>      | prod | i-0368a9527a5666fdf                                | Running        |

So in the code we create the ec2 client and also the ec2.resource along with the instance id

```

1 import boto3
2
3 ec2_client = boto3.client(*args: 'ec2', region_name="eu-west-2")
4 ec2_resource = boto3.resource(*args: 'ec2', region_name="eu-west-2")
5
6 instance_id = "i-0368a9527a5666fdf"
7
8 You, Moments ago • Uncommitted changes

```

Now we will collect the volumes attached to that instance Id which in this case is one. We will use the .client and the function .describe\_volumes()

And we will use the filter 'attachment.instance-id' to filter the volumes that belong to that specific instance

#### • Filters (list) –

The filters.

- `attachment.attach-time` - The time stamp when the attachment initiated.
- `attachment.delete-on-termination` - Whether the volume is deleted on instance termination.
- `attachment.device` - The device name specified in the block device mapping (for example, `/dev/sda1`).
- `attachment.instance-id` - The ID of the instance the volume is attached to.
- `attachment.status` - The attachment state ( `attaching` | `attached` | `detaching` ).
- `availability-zone` - The Availability Zone in which the volume was created.
- `availability-zone-id` - The ID of the Availability Zone in which the volume was created.

And here is the code so far:

```

1  import boto3
2
3  ec2_client = boto3.client(*args: 'ec2', region_name="eu-west-2")
4  ec2_resource = boto3.resource(*args: 'ec2', region_name="eu-west-2")
5
6  instance_id = "i-0368a9527a5666fdf"
7
8  ec2_client.describe_volumes(
9  Filters=[
10     {
11         'Name': 'attachment.instance-id',
12         'Values': [
13             'string',
14         ]
15     },
16 ],
17 )

```

And we pass the value which is the hardcoded instance\_id

```

1  import boto3
2
3  ec2_client = boto3.client(*args: 'ec2', region_name="eu-west-2")
4  ec2_resource = boto3.resource(*args: 'ec2', region_name="eu-west-2")
5
6  instance_id = "i-0368a9527a5666fdf"
7
8  ec2_client.describe_volumes(
9      Filters=[
10         {
11             'Name': 'attachment.instance-id',
12             'Values': [
13                 instance_id,
14             ]
15         },
16     ],
17 )
18
19

```

Now from the list of volumes attached to my EC2 instance I get the first one, we only have one at the moment and we are pretending that one has corrupted data.

-> instance\_volume = volumes['Volumes'][0]

```
restore-volume.py x
6 instance_id = 'i-0000727000007a1'
7
8 volumes = ec2_client.describe_volumes(
9     Filters=[
10         {
11             'Name': 'attachment.instance-id',
12             'Values': [
13                 instance_id,
14             ]
15         },
16     ],
17 )
18
19 instance_volume = volumes['Volumes'][0]
20 print(instance_volume)
21
```

Run restore-volume x

False, 'HiddenByDefault': False}, 'VolumeId': 'vol-0c6581b52a8530ed5', 'Size': 8, 'Snapshot

And the volumeId is the one we expect

Instances (1/2) [Info](#)

Saved filter sets  
Choose filter set ▼

| <input type="checkbox"/>            | Name | Instance ID         | Instance stat |
|-------------------------------------|------|---------------------|---------------|
| <input type="checkbox"/>            | dev  | i-0b874cf60fbd56050 | Running       |
| <input checked="" type="checkbox"/> | prod | i-0368a9527a5666fdf | Running       |

i-0368a9527a5666fdf (prod)

Details

Status and alarms

Monitoring

Security

▼ Root device details

Root device name  
 /dev/xvda

▼ Block devices

| <input checked="" type="checkbox"/> | Volume ID                             | Device name | Volum |
|-------------------------------------|---------------------------------------|-------------|-------|
| <input checked="" type="checkbox"/> | <a href="#">vol-0c6581b52a8530ed5</a> | /dev/xvda   | 8     |

So now we want to grab the latest snapshot of that volume to recover our instance.

And we filter the snapshots related to the corrupted volume:

```

19     instance_volume = volumes['Volumes'][0]
20     print(instance_volume)
21
22     snapshots = ec2_client.describe_snapshots(    You, A minute ago • Uncommitted
23         OwnerIds=['self'],
24         Filters=[
25             {
26                 'Name': 'volume-id', # from describe_snapshots()
27                 'Values': [
28                     instance_volume['VolumeId'], # from describe_volumes()
29                 ]
30             },
31         ]
32     )
33

```

And we need to sort the list of snapshots to get the latest.

```

restore-volume.py × cleanup-snapshots.py
22     print(instance_volume)
23
24     snapshots = ec2_client.describe_snapshots(
25         OwnerIds=['self'],
26         Filters=[
27             {
28                 'Name': 'volume-id', # from describe_snapshots()
29                 'Values': [
30                     instance_volume['VolumeId'], # from describe_volumes()
31                 ]
32             },
33         ]
34     )
35
36     latest_snapshot = sorted(snapshots['Snapshots'], key=itemgetter('StartTime'), reverse=True)

```

And we want the first element from the descending list which is the latest snapshot the most recent

-> latest\_snapshot = sorted(snapshots['Snapshots'],  
key=itemgetter('StartTime'), reverse=True)[0]

```

35
36 ✦ latest_snapshot = sorted(snapshots['Snapshots'], key=itemgetter('StartTime'), reverse=True)[0]
37
38

```

And let's see what we got:

```
restore-volume.py x cleanup-snapshots.py
30
31     },
32 ]
33 )
34
35 latest_snapshot = sorted(snapshots['Snapshots'], key=itemgetter('StartTime'), reverse=True)[0]
36 print(latest_snapshot['StartTime'])
37
38
```

Run restore-volume x

/Users/astrid/PythonProject/Automation\_with\_python/.venv/bin/python /Users/astrid/PythonProject/Automa  
2026-07-09 09:17:17.864000+00:00

Process finished with exit code 0

And it is what we expect

Snapshots (1/2) Info

Last updated 29 minutes ago [Recycle Bin](#) [Actions](#)

Snapshot scope: Owned by me

|                                     | Name | Snapshot ID            | Full snapshot size | Volume size | Description | Storage tier | Snapshot status | Started                |
|-------------------------------------|------|------------------------|--------------------|-------------|-------------|--------------|-----------------|------------------------|
| <input checked="" type="checkbox"/> |      | snap-07fe9606323507658 | 1.64 GiB           | 8 GiB       | –           | Standard     | Completed       | 2026/07/09 10:17 GMT+1 |
| <input type="checkbox"/>            |      | snap-0771892236d43e783 | 1.64 GiB           | 8 GiB       | –           | Standard     | Completed       | 2026/07/09 10:16 GMT+1 |

Next, let's create a new volume with the snapshot and for that we use the function `.create_volume()`

```
35 latest_snapshot = sorted(snapshots['Snapshots'], k
36 print(latest_snapshot['StartTime'])
37
38 ec2_client.create_volume(
39
40 )
```

And we can pass the parameter 'SnapshotId'

# create\_volume

ON THIS PAGE  
EC2.Client.create\_volume()

**EC2.Client.create\_volume(\*\*kwargs)**

Creates an EBS volume that can be attached to an instance in the same Availability Zone.

You can create a new empty volume or restore a volume from an EBS snapshot. Any Amazon Web Services Marketplace product codes from the snapshot are propagated to the volume.

You can create encrypted volumes. Encrypted volumes must be attached to instances that support Amazon EBS encryption. Volumes that are created from encrypted snapshots are also automatically encrypted. For more information, see [Amazon EBS encryption](#) in the *Amazon EBS User Guide*.

You can tag your volumes during creation. For more information, see [Tag your Amazon EC2 resources](#) in the *Amazon EC2 User Guide*.

For more information, see [Create an Amazon EBS volume](#) in the *Amazon EBS User Guide*.

See also: [AWS API Documentation](#)

## Request Syntax

```
response = client.create_volume(
    AvailabilityZone='string',
    AvailabilityZoneId='string',
    Encrypted=True|False,
    Iops=123,
    KmsKeyId='string',
    OutpostArn='string',
    Size=123,
    SnapshotId='string',
    VolumeType='standard'|'io1'|'io2'|'gp2'|'sc1'|'st1'|'gp3',
    TagSpecifications=[
        {
            'ResourceType': 'capacity-reservation'|'client-vpn-endpoint'|'customer-gatew
```

Here is the code so far:

```
34
35 latest_snapshot = sorted(snapshots['Snapshots'], key=itemgetter('StartTime'), reverse=True)[0]
36 print(latest_snapshot['StartTime'])
37
38 ec2_client.create_volume(
39     SnapshotId=latest_snapshot['SnapshotId']
40 )
```

And we also will pass the AZ of the instance

Instances (1/2) Info Last updated about 1 hour ago

Saved filter sets  
Choose filter set

| <input type="checkbox"/>            | Name | Instance ID         | Instance state | Instance type | Status check | Availability Zone | Public |
|-------------------------------------|------|---------------------|----------------|---------------|--------------|-------------------|--------|
| <input type="checkbox"/>            | dev  | i-0b874cf60fbd56050 | Running        | t3.micro      | Initializing | eu-west-2c        | ec2-18 |
| <input checked="" type="checkbox"/> | prod | i-0368a9527a5666fdf | Running        | t3.micro      | Initializing | eu-west-2c        | ec2-13 |

```
38 ec2_client.create_volume(
39     SnapshotId=latest_snapshot['SnapshotId'],
40     AvailabilityZone="eu-west-2c" # this can be improved by getting the AZ from the instance
41 )
42
```

When we created the EC2 instances we tagged them and then we tagged the volume attached to the 'prod' instance with 'prod' as well, so the new volume needs the tag:

## Request Syntax

```
response = client.create_volume(  
    AvailabilityZone='string',  
    AvailabilityZoneId='string',  
    Encrypted=True|False,  
    Iops=123,  
    KmsKeyId='string',  
    OutpostArn='string',  
    Size=123,  
    SnapshotId='string',  
    VolumeType='standard'|'io1'|'io2'|'gp2'|'sc1'|'st1'|'gp3',  
    TagSpecifications=[  
        {  
            'ResourceType': 'capacity-reservation'|'client-vpn-endpoint'|'customer-gatew  
            'Tags': [  
                {  
                    'Key': 'string',  
                    'Value': 'string'  
                },  
            ],  
        },  
    ],  
    MultiAttachEnabled=True|False,  
    Throughput=123,  
    ClientToken='string',  
    VolumeInitializationRate=123,  
    Operator={  
        'Principal': 'string'  
    },  
    DryRun=True|False  
)
```

So I copy the key and its structure

```
38     ec2_client.create_volume(  
39         SnapshotId=latest_snapshot['SnapshotId'],  
40         AvailabilityZone="eu-west-2c", # this can be improved by getting the AZ from the instance  Yo  
41         TagSpecifications = [  
42             {  
43                 'ResourceType': 'capacity-reservation' | 'client-vpn-endpoint' | 'customer-gateway' |  
44                 'Tags': [  
45                     {  
46                         'Key': 'string',  
47                         'Value': 'string'  
48                     },  
49                 ],  
50             },  
51         ],  
52     )  
53
```

And I edit it

```

38     ec2_client.create_volume(
39         SnapshotId=latest_snapshot['SnapshotId'],
40         AvailabilityZone="eu-west-2c", # this can be improved by getting the AZ from the instance
41         TagSpecifications = [
42             {
43                 'ResourceType': 'volume',
44                 'Tags': [
45                     {
46                         'Key': 'Name',
47                         'Value': 'prod'
48                     },
49                 ]
50             },
51         ],
52     )
53

```

Now let's attach the new volume to the EC2 instance

So we use the `.resource` and EC2 has a resource Called `Instance()` and we pass the instance id that we hardcoded so no need to iterate using `.client` we can use `.resource` directly because of the id.

<https://docs.aws.amazon.com/boto3/latest/reference/services/ec2/instance/>

## Resources

Resources are available in boto3 via the `resource` method. For more detailed instructions and examples on the usage of resources, see the [resources user guide](#).

The available resources are:

- [Service Resource](#)
- [ClassicAddress](#)
- [DhcpOptions](#)
- [Image](#)
- [Instance](#)
- [InternetGateway](#)
- [KeyPair](#)
- [KeyPairInfo](#)
- [NetworkAcl](#)

ON THIS PAGE

- Client
  - EC2.Client
- Paginators
- Writers
- [Resources](#)

EC2 / Resource / Instance

# Instance

 **Note**

Before using anything on this page, please refer to the resources [user guide](#) for the most recent guidance on using resources.

**class EC2.Instance(id)**

A resource representing an Amazon Elastic Compute Cloud (EC2) Instance:

```
import boto3

ec2 = boto3.resource('ec2')
instance = ec2.Instance('id')
```

**PARAMETERS:**

- id** (*string*) – The Instance's id identifier. This **must** be set.

## Identifiers

Identifiers are properties of a resource that are set upon instantiation of the resource. For more information about identifiers refer to the [Resources Introduction Guide](#).

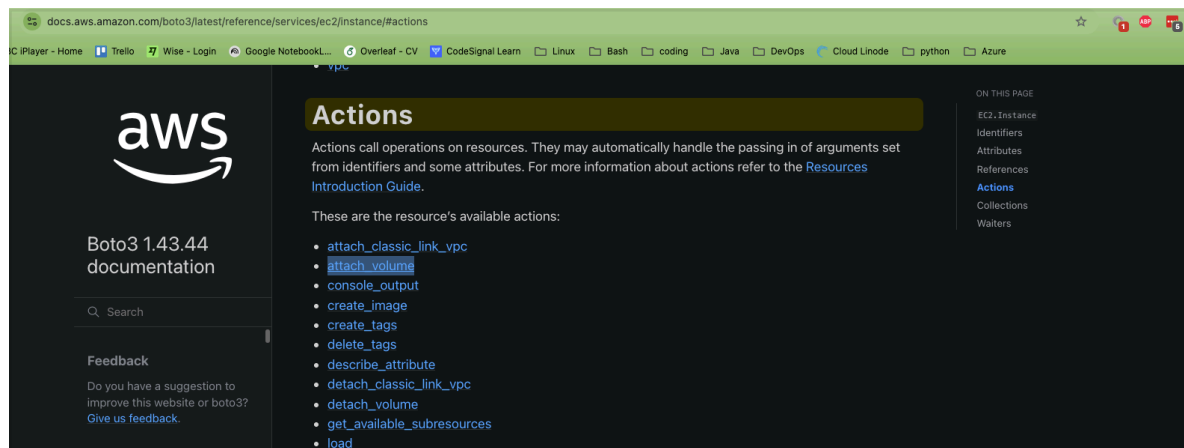
These are the resource's available identifiers:

- [id](#)

And here is the code so far  
-> `ec2_resource.Instance(instance_id)`

```
38 ec2_client.create_volume(  
39     SnapshotId=latest_snapshot['SnapshotId'],  
40     AvailabilityZone="eu-west-2c", # this can be improved by getting the AZ from the instance  
41     TagSpecifications = [  
42         {  
43             'ResourceType': 'volume',  
44             'Tags': [  
45                 {  
46                     'Key': 'Name',  
47                     'Value': 'prod'  
48                 },  
49             ],  
50         },  
51     ],  
52 )  
53  
54 ec2_resource.Instance(instance_id) You, 5 minutes ago • Uncommitted changes
```

And Instance Resource has a function called `attach_volume()`



[https://docs.aws.amazon.com/boto3/latest/reference/services/ec2/instance/attach\\_volume.html](https://docs.aws.amazon.com/boto3/latest/reference/services/ec2/instance/attach_volume.html)

# attach\_volume

**EC2.Instance.attach\_volume(\*\*kwargs)**

Attaches an Amazon EBS volume to a `running` or `stopped` instance, and exposes it to the instance with the specified device name.

**Note**

The maximum number of Amazon EBS volumes that you can attach to an instance depends on the instance type. If you exceed the volume attachment limit for an instance type, the attachment request fails with the `AttachmentLimitExceeded` error. For more information, see [Instance volume limits](#).

After you attach an EBS volume, you must make it available for use. For more information, see [Make an EBS volume available for use](#).

If a volume has an Amazon Web Services Marketplace product code:

- The volume can be attached only to a stopped instance.
- Amazon Web Services Marketplace product codes are copied from the volume to the instance.
- You must be subscribed to the product.
- The instance type and operating system of the instance must support the product. For example, you can't detach a volume from a Windows instance and attach it to a Linux instance.

For more information, see [Attach an Amazon EBS volume to an instance](#) in the *Amazon EBS User Guide*.

See also: [AWS API Documentation](#)

## Request Syntax

```
response = instance.attach_volume(
    Device='string',
    VolumeId='string',
    EbsCardIndex=123,
    DryRun=True|False
)
```

And Device and VolumeId are required parameters

PARAMETERS:

- **Device** (*string*) –  
**[REQUIRED]**  
The device name (for example, `/dev/sdh` or `xvdh`).
- **VolumeId** (*string*) –  
**[REQUIRED]**  
The ID of the EBS volume. The volume and instance must be within the same Availability Zone.
- **EbsCardIndex** (*integer*) – The index of the EBS card. Some instance types support multiple EBS cards. The default EBS card index is 0.
- **DryRun** (*boolean*) – Checks whether you have the required permissions for the action, without actually making the request, and provides an error response. If you have the required permissions, the error response is `DryRunOperation`. Otherwise, it is `UnauthorizedOperation`.

So we pass the argument for volumeId in line 55. First we store the response of `.create_volume()` from line 38 🙌

```
37
38 new_volume = ec2_client.create_volume(  You, Moments ago • Uncommitted changes
39     SnapshotId=latest_snapshot['SnapshotId'],
40     AvailabilityZone="eu-west-2c", # this can be improved by getting the AZ from the instance
41     TagSpecifications = [
42         {
43             'ResourceType': 'volume',
44             'Tags': [
45                 {
46                     'Key': 'Name',
47                     'Value': 'prod'
48                 },
49             ]
50         },
51     ],
52 )
53
54 ec2_resource.Instance(instance_id).attach_volume(
55     VolumeId=new_volume['VolumeId']
56 )
```

And then we pass the argument for Device

PARAMETERS:

- **Device** (*string*) –  
**[REQUIRED]**  
The device name (for example, `/dev/sdh` or `xvdh`).
- **VolumeId** (*string*) –  
**[REQUIRED]**  
The ID of the EBS volume. The volume and instance must be within the same Availability Zone.

We will use the one from our faulty or corrupted volume -> `/dev/xvda`

**Volumes (1/2)** [Info](#)

Saved filter sets  
Choose filter set ▼

Search

| <input type="checkbox"/>            | Name <a href="#">✎</a> | Volume ID             | Type | Size  | IOPS | Throughput |
|-------------------------------------|------------------------|-----------------------|------|-------|------|------------|
| <input checked="" type="checkbox"/> | prod                   | vol-0c6581b52a8530ed5 | gp3  | 8 GiB | 3000 | 125        |
| <input type="checkbox"/>            |                        | vol-000a63cefe7025af1 | gp3  | 8 GiB | 3000 | 125        |

**Volume ID: vol-0c6581b52a8530ed5 (prod)**

Details
Status checks
Monitoring
Tags

**Volume ID**  
 vol-0c6581b52a8530ed5 (prod)

**AWS Compute Optimizer finding**  
 Opt-in to AWS Compute Optimizer for recommendations. | [Learn more](#)

**Fast snapshot restored**  
No

**Attached resources**  
i-0368a9527a5666fdf (prod)  
(EBS card index: 0): [/dev/xvda](#) (attached)

**Source**

**Size**  
 8 GiB

**Volume state**  
 In-use

**Availability Zone**  
euw2-az1 (eu-west-2c)

**Outposts ARN**  
-

And we will edit the name a little bit to -> /dev/xvdb noted I changed a for b at the end.

So now the name is unique /dev/xvdb for the device in the same EC2 instance and it will be accepted by the EC2 instance.

## — tangent: Device —

Q:

The EC2 instance current volume has this device name '/dev/xvda' so we will pass the argument '/dev/xvdb'.

1. what is a device?
2. also I have 2 servers each with a different volume but both have the same device name why this is ok?

|   | Attached resources                               | ▼ | Sta |
|---|--------------------------------------------------|---|-----|
| + | i-0368a9527a5666fdf (prod): /dev/xvda (attached) |   | ✓   |
| + | i-0b874cf60fbd56050 (dev): /dev/xvda (attached)  |   | ✓   |
|   |                                                  |   |     |
|   |                                                  |   |     |

A:

**First question** -> what is a device?

Think of an EC2 instance like a computer, and "device name" like a USB port label.

When you plug in a hard drive (an EBS volume) to a computer, the computer needs to know where to plug it in — which "slot" to use. Device is just that slot label.

- /dev/xvda = the slot where your instance's main hard drive (the one with the operating system) is already plugged in. It's occupied.
- /dev/xvdb = an empty slot. When you attach a new volume, you tell AWS "plug it into slot b" — hence /dev/xvdb.

You can't tell AWS "plug the new drive into slot a" because slot a is already taken (by the root volume). That's why your teacher's example uses /dev/xvdb — it's just the next free slot on that instance.

Q:

so the hard drive gets plug to what? the motherboard? is here where slots are?

A:

In a real physical computer:

Yes, hard drives plug into the motherboard, into slots called SATA ports (or older IDE ports). The motherboard has a disk controller that manages these ports. When the OS boots up, it looks at each port, finds a disk, and gives it a name like /dev/sda (port 1), /dev/sdb (port 2), etc. The port number roughly determines the device name.

In EC2 — it's virtual, so there's no real motherboard

An EC2 instance isn't a physical machine — it's a virtual computer. The actual EBS volume (your "hard drive") isn't even in the same physical box; it lives on separate storage servers elsewhere in AWS's data center, connected over the network.

But AWS pretends there's a motherboard, for the OS's sake. It emulates a virtual disk controller inside your instance, with virtual "slots" (called device names). When you call `attach_volume` with `Device="/dev/xvdb"`, you're telling AWS:

■ "Take this remote network-based volume, and make it appear to the instance's operating system as if it's a real disk plugged into slot xvdb."

The OS inside the instance has no idea the disk is actually remote — it just sees a new disk show up at `/dev/xvdb`, exactly like it would if you'd physically plugged something into a SATA port.

So to directly answer "plugged into what?":

Not a real motherboard — a virtual disk controller that AWS's hypervisor presents to your instance. The device name is the label for the virtual slot on that virtual controller.

That's also why device names don't need to be unique across instances (going back to your earlier question) — each instance has its own separate virtual controller with its own separate set of slots.

**Now, your second question:** why can two different servers both use "slot b" (`/dev/xvdb`)?

Because each computer (instance) has its own set of slots. Server 1's "slot b" and Server 2's "slot b" are two totally different, unrelated slots — they just happen to have the same label, on two different machines.

It's like two houses both having a room called "Bedroom 2." The fact that they share a name doesn't mean anything — they're in different houses. Similarly:

- Instance A's `/dev/xvdb` → its own volume
- Instance B's `/dev/xvdb` → a completely different volume

AWS doesn't compare device names across instances — only within the same instance does it care that names aren't reused. So there's no conflict at all.

The one rule to remember: within a single instance, every attached volume needs a unique device name. Across different instances, it doesn't matter — reuse away.

Q:

thanks! this makes more sense. in the real world I image not all of the SATA ports are used, so when the OS looks for each port and there is no disk then it doesn't name it?

A:

Exactly right. The OS only assigns a device name to ports that actually have something plugged in. An empty SATA port gets skipped entirely — no `/dev/sdX` name is created for it. So if a motherboard has 6 SATA ports but only 2 have drives connected (say, port 1 and port 3), the OS would name them `/dev/sda` and `/dev/sdb` — the naming is based on what's found, not on the physical port number.

This maps back to EC2 nicely: when your instance boots, it only has `/dev/xvda` (root volume) because that's the only "slot" with something attached. The moment you call `attach_volume` with `Device="/dev/xvdb"`, that's like plugging a new drive into an empty port live — and the OS inside the instance detects it and the name `/dev/xvdb` becomes real/visible (sometimes you may need to run a command like `lsblk` to see it show up, since it's a live/hot-plug event rather than something found at boot).

Q:

and can I give it any name for the EC2? or I need to follow the `xvda`, `xvdb`, `xvdc` pattern?

A:

Not any name — AWS restricts you to specific patterns, and it depends on your instance type.

For Linux instances (the classic pattern):

AWS recommends using `/dev/sdf` through `/dev/sdp` for additional (non-root) volumes. So valid options look like:

- `/dev/sdf`, `/dev/sdg`, `/dev/sdh`, ... `/dev/sdp`

You'll often see these referred to as `/dev/xvdf`, `/dev/xvdg`, etc. — that's because on many Linux AMIs, the OS automatically renames `sdX` → `xvdX` internally (a quirk of Xen virtualization). So `/dev/sdf` you request often shows up as `/dev/xvdf` inside the instance. That's why your teacher's example used `/dev/xvdb` directly — some setups let you specify the `xvd` name outright too.

Reserved / don't use:

- `/dev/sda1` (or `xvda`) — reserved for the root volume
- `/dev/sdb` through `/dev/sde` — reserved by AWS for internal use on some instance types (historically used for instance store volumes)

So in practice, takeaway: for extra volumes, stick to `/dev/sdf`–`/dev/sdp`, don't invent your own name.

One more wrinkle (newer instance types):

On modern "Nitro-based" instances (most current-gen types, e.g. `t3`, `m5`, `c6`),

EBS volumes actually attach as NVMe devices, and the OS assigns its own name like `/dev/nvme1n1` — ignoring whatever device name you passed! The name you specify still gets stored as a tag/hint on the volume, but the real device path inside the OS is different. This trips people up a lot — if `lsblk` doesn't show `/dev/xvdb`, check for an nvme device instead.

So: you have a small set of allowed names to choose from (not truly "any" name), and on newer instance types the name you pick may not even be the one the OS actually uses.

## — end tangent —

So the code is:

```
53
54     ec2_resource.Instance(instance_id).attach_volume(
55         VolumeId=new_volume['VolumeId'],
56         Device='/dev/xvdb'
57     )
```

Now we create the volume in line 38 and then in line 54 we attached the volume to the EC2 instance, but creating the volume takes some time until it is in an **available** state.

Once the volume is in a AVAILABLE state, then the volume can be attached to the EC2 instance so we have a TIMING issue.



**Volume state**

Volume state describes the availability of an Amazon EBS volume. You can view the volume state in the **State** column by using the [describe-volumes](#) AWS CLI command.

The possible volume states are:

- creating**  
The volume is being created.
- available**  
The volume is not attached to an instance.
- in-use**  
The volume is attached to an instance.
- deleting**  
The volume is being deleted.
- deleted**  
The volume is deleted.
- error**  
The underlying hardware related to your EBS volume has failed, and the data associated with the volume is at risk. To restore the volume or recover the data on the volume, see [My EBS volume has a status of "error"](#).

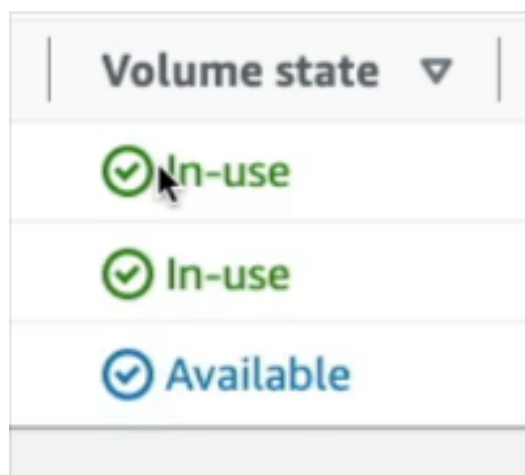
So the code as it is will throw an error:

```
"/Users/nana/PycharmProjects/my-python-project/venv/lib/python3.12/site-packages/botocore/client.py", line 983, in _make_api_call
    raise error_class(parsed_response, operation_name)
botocore.exceptions.ClientError: An error occurred (IncorrectState) when calling the AttachVolume operation: vol-0f5ebcd3ed12fb458 is not 'available'.
```

Saying that the new volume is not available !!

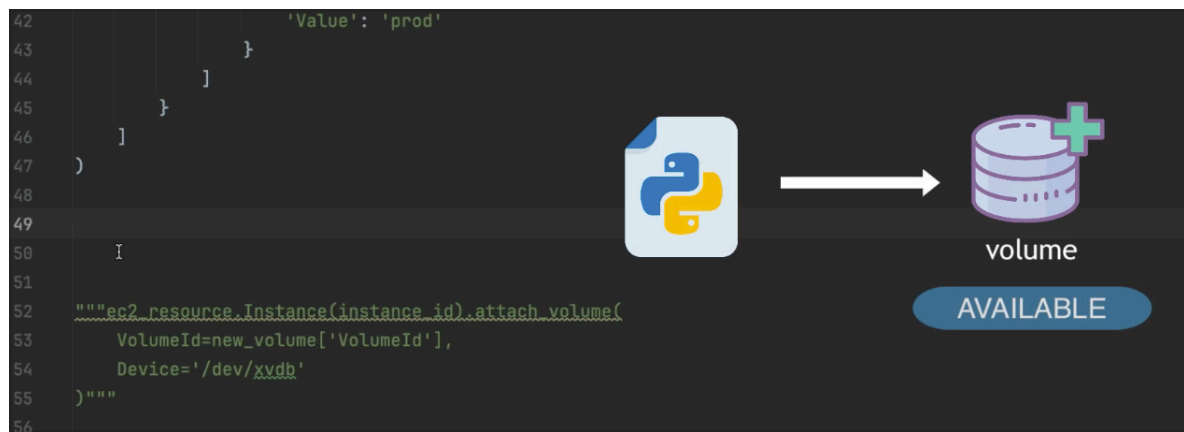
It is still in the create state when we try to attach it.

Then if we check the AWS console the volume is now available, it was a better of few milliseconds

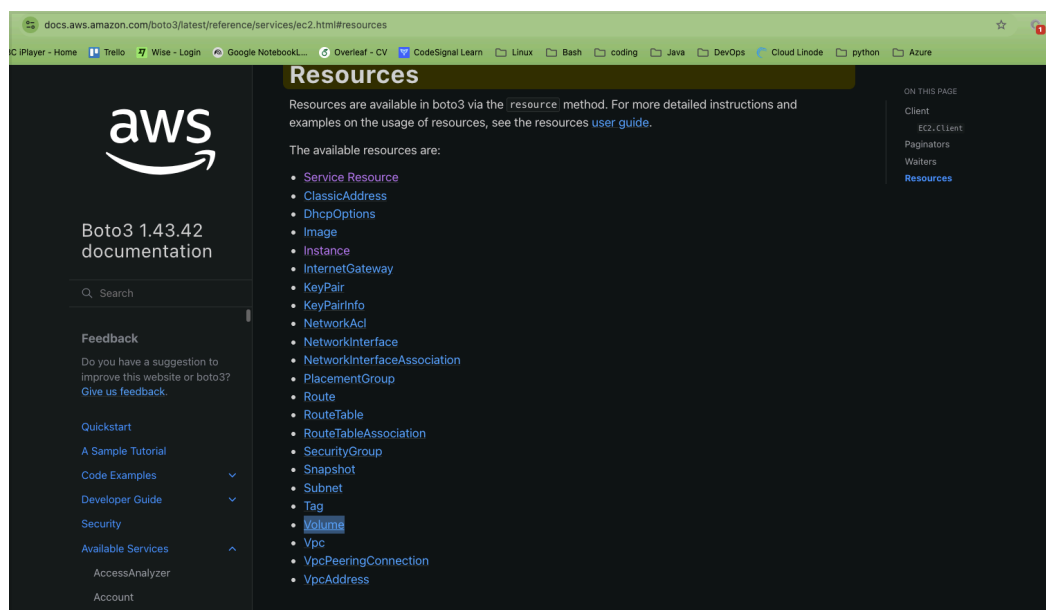


And because we couldn't attach it is not 'In-use' state.

So one way to fix the timing issue is to check the volume until is AVAILABLE

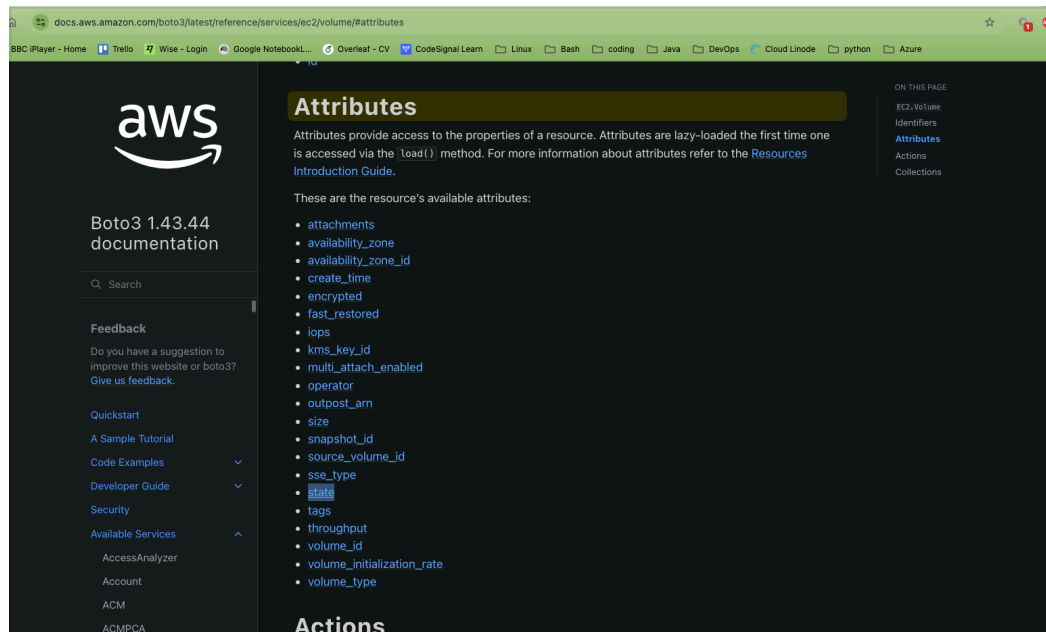


So using .resource EC2 has a Volume resource



And the volume resource has an attribute called 'state'

<https://docs.aws.amazon.com/boto3/latest/reference/services/ec2/volume/state.html>



So the code goes like this:

```

53
54     while True:
55         vol = ec2_resource.Volume(new_volume['Volume'])
56         print(vol.state)
57         if vol.state == 'available':
58             💡
59     You, 6 minutes ago • Uncommitted changes

```

And when is available then we attach the volume to the EC2 instance, so now we add the attachment code inside the IF statement

```

54     while True:
55         vol = ec2_resource.Volume(new_volume['Volume'])
56         print(vol.state)
57         if vol.state == 'available':
58             ✨ ec2_resource.Instance(instance_id).attach_volume(
59                 VolumeId=new_volume['VolumeId'],
60                 Device='/dev/xvdb'
61             )
62         break
63

```

And break the loop when the attachment is done!

Now we execute the code:

```
Automation_with_python  Module_14/Automate_Python_chapter-11  restore-volume  ⏏ ⚙ ⋮
restore-volume.py  cleanup-snapshots.py
52
53
54     while True:
55         vol = ec2_resource.Volume(new_volume['VolumeId'])
56         print(vol.state)
57         if vol.state == 'available':
58             ec2_resource.Instance(instance_id).attach_volume(
59                 VolumeId=new_volume['VolumeId'],
60                 Device='/dev/xvdb'
61             )
62             break
63
Run  restore-volume  x
⏏ ⚙ ⋮
↑ creating
↓ creating
⇅ creating
⇅ creating
🖨 creating
🗑 creating
available
Process finished with exit code 0
```

And we check the AWS console

Volumes (1/3) Info Last updated 1 minute ago Recycle Bin Actions Create volume

Saved filter sets Choose filter set

|                                     | Name | Volume ID             | T... | S...  | I... | T... | S...    | S... | Crea...   | A...  | Volume state | Alarm st... | Attached resources                               |
|-------------------------------------|------|-----------------------|------|-------|------|------|---------|------|-----------|-------|--------------|-------------|--------------------------------------------------|
| <input checked="" type="checkbox"/> | prod | vol-047be4383ce74639d | gp2  | 8 GiB | 100  | -    | snap... | -    | 2026/0... | eu... | In-use       | No alarms   | I-0368a9527a5666fdf (prod): /dev/xvdb (attached) |
| <input type="checkbox"/>            | prod | vol-0c6581b52a8530ed5 | gp3  | 8 GiB | 3000 | 125  | snap... | -    | 2026/0... | eu... | In-use       | No alarms   | I-0368a9527a5666fdf (prod): /dev/xvda (attached) |
| <input type="checkbox"/>            |      | vol-000a63cfe7025af1  | gp3  | 8 GiB | 3000 | 125  | snap... | -    | 2026/0... | eu... | In-use       | No alarms   | I-0b874cf60fbd56050 (dev): /dev/xvda (attached)  |

And we got the new volume attached to the EC2 instance and also it has the tag 'prod'

And if we go to the instance we can see that it has 2 volumes attached 🤔

Instances (1/2) [Info](#)

Last updated about 2 hours ago [Refresh](#) [Connect](#)

Saved filter sets  
Choose filter set

Find Instance by attribute or tag (case-sensitive)

| <input type="checkbox"/>            | Name | Instance ID         | Instance state       | Instance type | Status check              | Availability Zone | Public IPv4 DNS  |
|-------------------------------------|------|---------------------|----------------------|---------------|---------------------------|-------------------|------------------|
| <input type="checkbox"/>            | dev  | i-0b874cf60fbd56050 | <span>Running</span> | t3.micro      | <span>Initializing</span> | eu-west-2c        | ec2-18-171-224-2 |
| <input checked="" type="checkbox"/> | prod | i-0368a9527a5666fdf | <span>Running</span> | t3.micro      | <span>Initializing</span> | eu-west-2c        | ec2-13-40-37-245 |

i-0368a9527a5666fdf (prod)

Details

Status and alarms

Monitoring

Security

Networking

Storage

Tags

▼ Root device details

Root device name  
/dev/xvda

Root device type  
EBS

EBS opt  
enabled

▼ Block devices

Filter block devices

| <input type="checkbox"/>            | Volume ID             | Device name | Volume size (GiB) | Volume State        | Attachment status     | EBS card index | Attachm |
|-------------------------------------|-----------------------|-------------|-------------------|---------------------|-----------------------|----------------|---------|
| <input checked="" type="checkbox"/> | vol-0c6581b52a8530ed5 | /dev/xvda   | 8                 | <span>In-use</span> | <span>Attached</span> | 0              | 2026/07 |
| <input type="checkbox"/>            | vol-047be4383ce74639d | /dev/xvdb   | 8                 | <span>In-use</span> | <span>Attached</span> | 0              | 2026/07 |